

Project Report

5G teleoperated Robotic Arm

David Rajchman

Table of contents

1	Introduction	2
2	Project assignment	2
2.1	Approach to solving the assignment	2
2.2	Additional devices	2
2.3	ROS2 modular offloading codebase	3
2.4	Original network topology	4
2.5	Modular Gateway	4
2.6	P2P mode - New	5
2.7	P2P with discovery	5
2.8	ROARM Manufacturer ROS2 Demo	5
2.9	Custom GUI	7
2.10	5G network	9
2.11	Testing results and performance	10
2.12	Future steps	10
3	Demo time	11
3.1	Hardware Demonstration	11
3.2	P2P mode setup	11
3.3	P2P DS mode setup	12

1 Introduction

The project is meant

2 Project assignment

The main goal of the project was for the author to familiarize themselves with the ROARM-M1 robotic arm, and create a solution that would allow for remote control of the robotic arm over the EURECOM 5G network. The author was expected to use the ROS2 toolchain for the robotic arm control.

2.1 Approach to solving the assignment

The author was given access to a raspberry pi 4 and a lab laptop running ubuntu 22 lts for the use in his project. The following connection scheme was created to allow for the assignment completion Figure 1 .

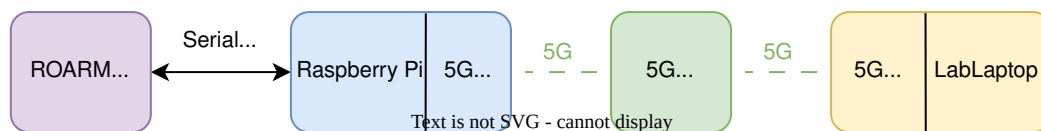
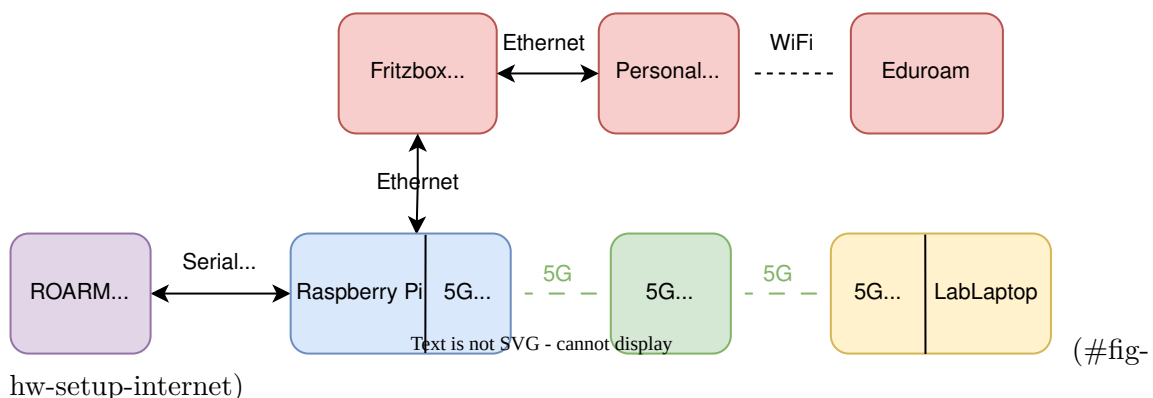


Figure 1: The physical elements of the solution

To make the completion of the project assignment more straightforward, it was dissected into 3 distinct parts. 1) Connect the robotic arm to the ROS2 environment and allow for its control with the gamepad 1) Allow for forwarding of the control topics between two ros2 instances 1) Connect the arm-side ROS2 instance with the controller-side ROS2 instance together using the EURECOM 5G network

2.2 Additional devices



hw-setup-internet)

To allow for the configuration of the raspberry pi it had to be given internet access. Initially use of the lab LAN network was considered, however the ports have a MAC filter on them, which made connection of the PI complicated. As a second option the author used his personal laptop which had access to the EURECOM eduroam network. Then the author forwarded the network to the RPI. By default the ubuntu 24 LTS for raspberry does not allow for a direct connection between a laptop and the RPI, this is something that can be configured, but without ssh access, the configuration would have to be done by adjusting system files on the boot sdcard. This is a process which is prone to errors and very hard to debug. For this reason a router was used as a DHCP server and a switch and both the RPI and the personal laptop were connected to it. Then the RPI simply used the laptop IP as the default gateway. (configuration script available in appendix XXX). This configuration shown in Image ?@fig-hw-setup-internet also allows for reliable SSH access to the RPI. One last issues was the time, the RPI did not sync its time through NTP when its RTC time was greatly different from real time, this could be mitigated by manually setting approximately correct time and then the NTP would sync the precise time. A simple script was created to automatically get approximately accurate time from the internet

2.3 ROS2 modular offloading codebase

The author of this project has worked on another project with similar use in the 6G lab at FEE CTU. The resulting codebase fully written by the author of this project was used as a starting point for the ros2 topic forwarding in this project. This chapter will briefly describe the original codebase and its purpose to make the final project codebase easier to understand.

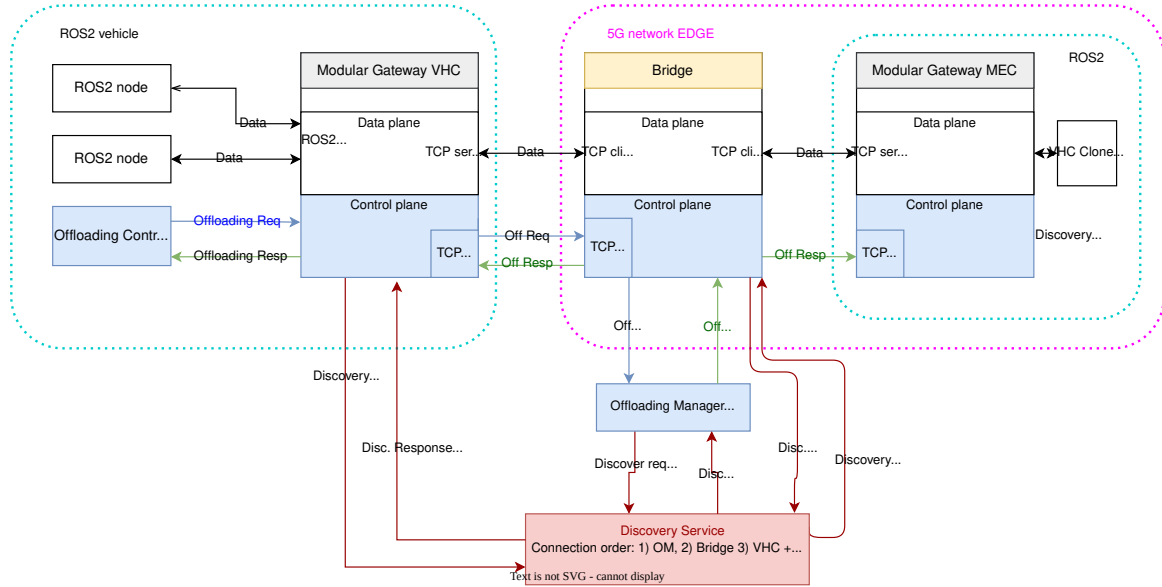
The goal of the 6G lab assignment was to create a solution that could forward data from ros2 topic on one ros2 instance to another ros2 instance. The intended usecase was computation offloading from vehicles to MEC servers. The solution had to be highly flexible, as one of the requirements was that a single vehicle had to be able to offload to at least 2 different MEC server at one time with every MEC server working on a different task. The MEC server were clones of the vehicle ROS2 environment. There are existing solutions allowing for connection of 2 existing ros2 environments together utilising the ros2 DDS protocol. However none of them provide this level of flexibility.

For this reason an architecture shown in @fig_om_network was designed. The main components were the modular gateway, bridge and offloading manager. The offloading manager held a configurable algorithm that distributed limited resources to the vehicles, by approving or rejecting the offloading requests. If the request was approved, the bridge would create an internal route for the offloaded topics between the vehicle and the assigned MEC server. The data flowed in a separate connection and in all components there was a clear distinction between data plane and control plane parts.

The modular gateway shown in @fig_mqw is the core part of the solution that transmits the data from ros2 topics towards the 5G network. Each topic has its own handler, this handler does ROS2 serialization and deserialization, and also allows for data manipulation for example removing unnecessary parts of the message. To achieve highest possible performance

the modular gateway is multithreaded. Detailed overview of the entire offloading architecture is available in the git repository documentation

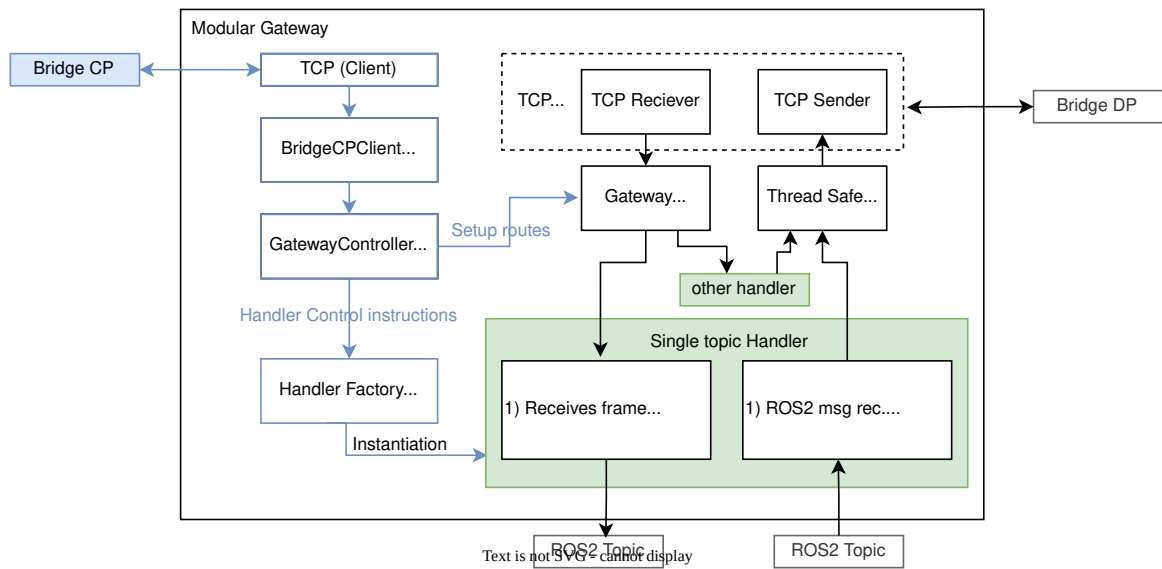
2.4 Original network topology



{fig_om_net-

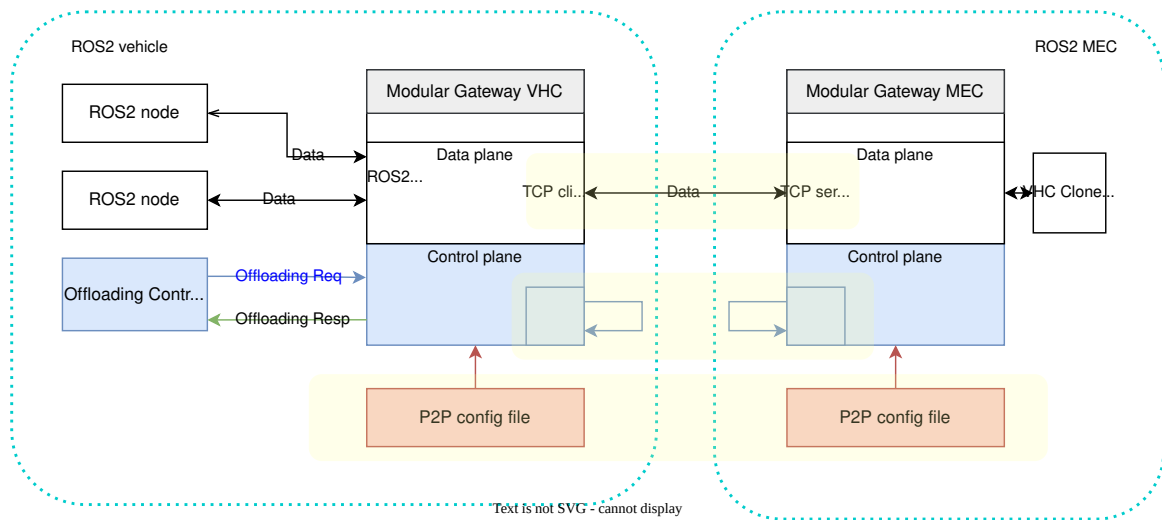
work}

2.5 Modular Gateway

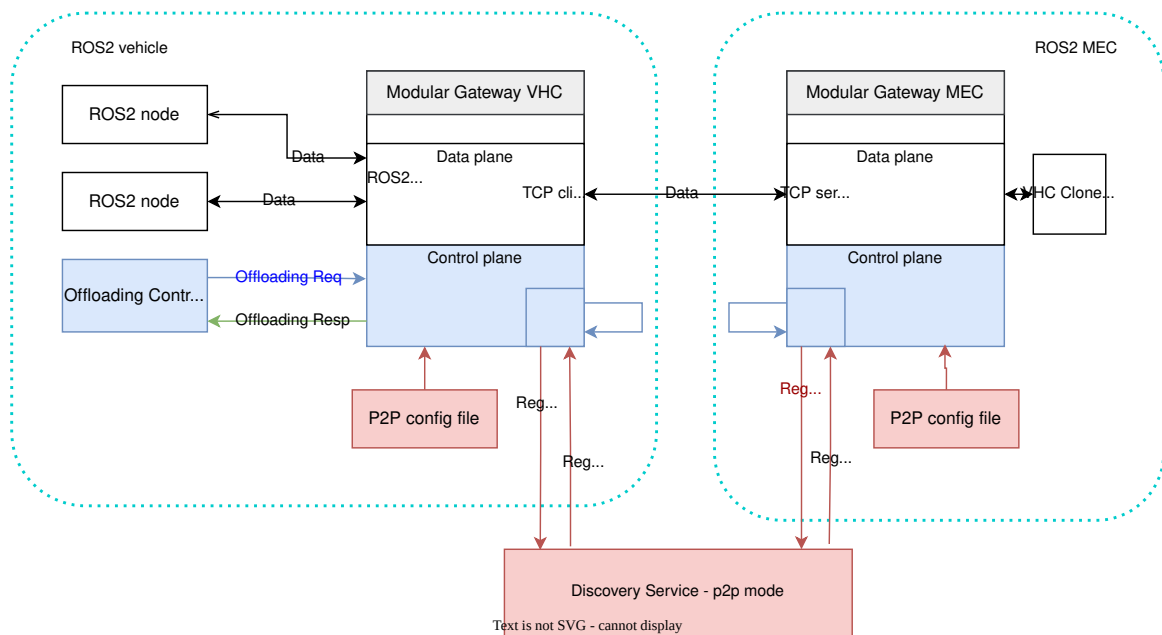


{fig_mqw}

2.6 P2P mode - New



2.7 P2P with discovery

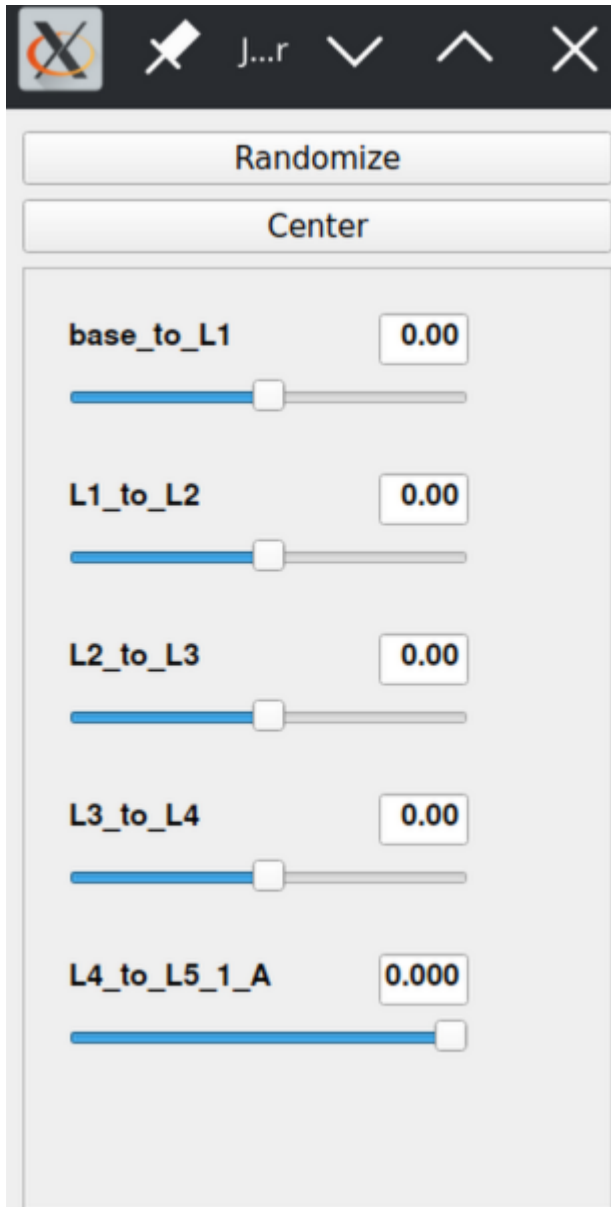


2.8 ROARM Manufacturer ROS2 Demo

The RoARM-M1 manufacturer provides a demo software that allows for successful connection of the roarm-m1. The software consists of 2 parts, first part which connects to the roarm-m1 using the serial interface over usb, and translates messages it receives on the `joint_states` topic to the serial communication protocol the ROARM uses. It also translates the radian

angle rotation values in the ros2 topic to a 12 bit digital value corresponding to the rotation. Second part of this demo is the graphical user interface which allows for manual joint angle adjustments using sliders. This interface node then publishes the joint state messages with the current value of the sliders.

- **joint_state_publisher_gui Node:**
 - Basic slider to manipulate each joint
- **/joint_states Topic:**
 - Intended joint coordinates (radian angle)
- **serial_ctrl Node:**
 - Subscribes to /joint_states,
 - Radians -> hardware servo values
 - Connects to the arm over serial

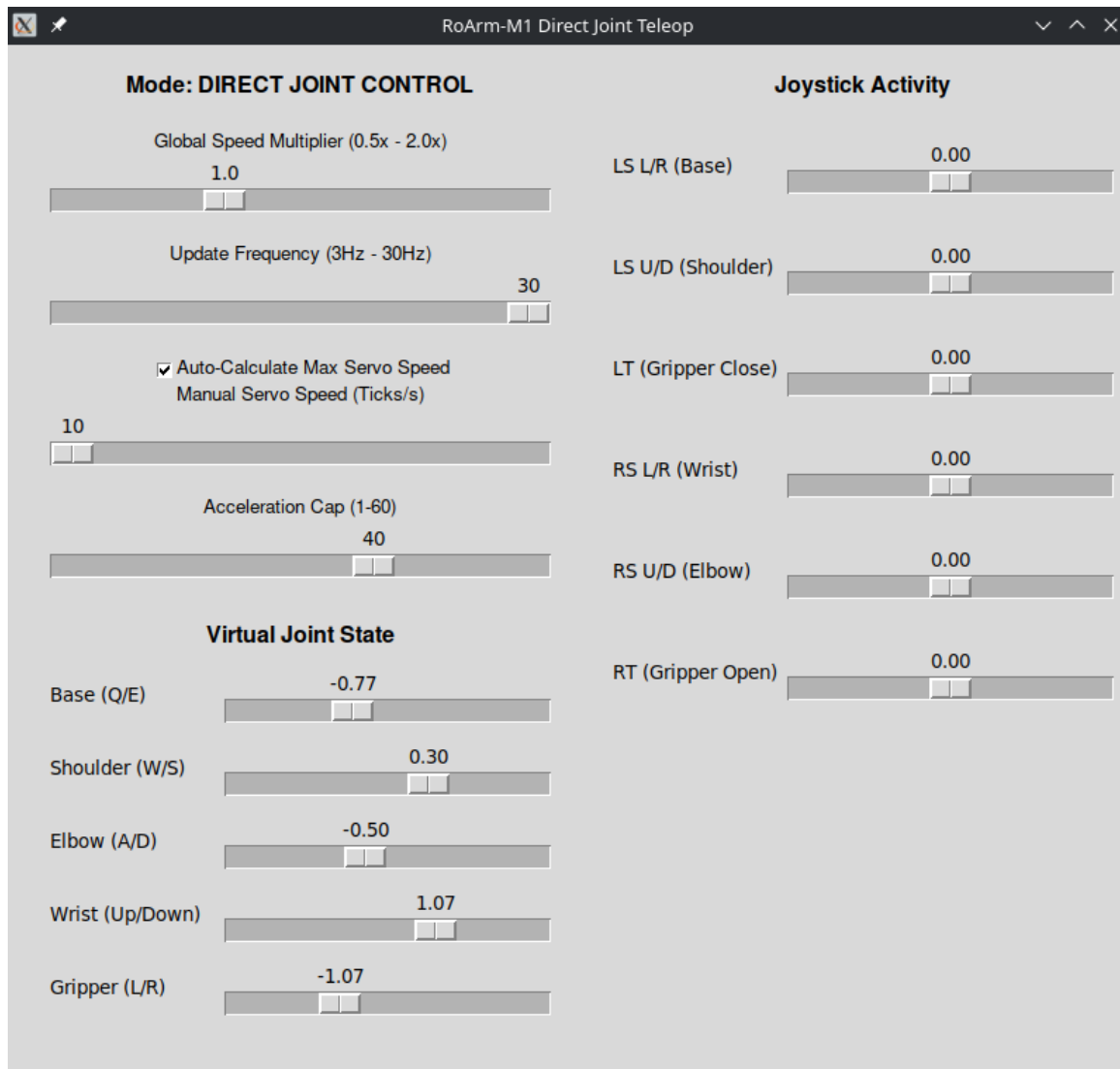


2.9 Custom GUI

To implement the necessary functionality a new graphical user interface node was developed. It allows for controls of the arm via either keyboard or a ros2 compatible gamepad such as the logitech F710. Since the 5G network introduces latency and jitter the interface decouples the control input from the dispatched position updates. The node holds a virtual joint state array, which is adjusted immediately upon an input is received, then the virtual joint state is dispatched in the messages at an adjustable frequency of 1 to 30 Hz. There are 3 additional adjustment sliders, the speed multiplier adjust the speed at which virtual joint state will be moved by the keyboard or the controller. The max movement speed limits the arm movement

speed and its value is forwarded allong with the required position on the `joint_state` topic. Lastly the acceleartion cap limits the maximum acceleration the arms joints can reach. Limiting the acceleration is supported by the ROARM controler firmware, however the driver which comunicates with the arm hardcodes the value to 60 steps/s^2 . For this reason the driver has been modified to send the acceleration maximum value received from the GUI node. The standard `joint_states` topic does not have a dedicated acceleration cap space, however the “effort” array was not used, so the acceleration cap values are being transmitted inside the “effort” array. To allows for control using the logitech gamepad, the `ros2 joy` package is used. this package connect to the gamepad and transmits the value of every axis on it to the `joy ros2` topic. Thus the GUI only needs to subscribe to the joy topic and then use the incoming messages. The arm has 5 Degrees of freedom, however the gamepad only has 4 axis on the joysticks, for this reason the griper arm was moved to the triggers, one trigger is used to open the gripper while the other is used to close the griper. the overall direction of travel of the griper is determined by the average of both trigger effects. thus it is not a problem if both triggers are presed at the same time

- Replaces `joint_state_publisher_gui`
- ROS2 joy node for gamepad support
- Keyboard control
- Adjustable
 - Speed
 - Joint Acceleration
 - Update frequency



2.10 5G network

- 5G OAI network created by Meriem
- Severe issues connecting
 - Probably due to interference
 - Only during working hours (stable after 5:30 pm)
- RTT 40ms between laptop and RPI
- RTT 50ms to Google DNS

2.11 Testing results and performance

The robotic arm has been repeatedly tested for ease of operation and fine control possibilities while connected both through a wired LAN network, and through the lab 5G network. In both scenarios very precise movements can be executed for example stacking 2 cans on top of each other. In the LAN controlled scenarios the acceleration cap and the update frequency do not result in changes to fine control if they are increased to their maximum range. When controlled over 5G high frequency high acceleration setting results in stuttery movement. This is caused due to the fact that the latency jitter will cause slight differences between the arrival times of each packet and if the update frequency is high, each update only contains a small movement change, thus for a high frequency those small movements can be received at an inconsistent timing which results in a stuttery movement pattern. This can be somewhat mitigated by decreasing the acceleration cap. However the most optimal control precision is achieved for frequencies around 15 Hz and acceleration limited to 20 steps/sec². The only downside of limiting the acceleration is the “floating effect” after the last update is received. This is caused due to the fact that the arm is slowly moving its final resting place and its movement is no longer affected by the newly coming position changes. Based on the testing it has been found that the acceleration cap is significantly better at limiting the arm stutters when being used on an unstable 5G network, the maximum movement speed cap is less effective and has a stronger floating effect than the acceleration cap if used. For this reason it is recommended to keep the maximum movement speed set at a high value and only adjust the acceleration cap and the update frequency

- ROS2 forwarding on localhost - ROS2 to ROS2 RTT ~ 1 ms
- 5G ROS2 to ROS2 RTT ~ 40 ms with jitter
- Precise arm control achievable
- Best settings in the GUI for smoothest motion:
 - 15 Hz update frequency
 - 20 steps/s² maximum acceleration (60 is default)

2.12 Future steps

- Slicing support The EURECOM network does not currently support 5G slicing, however this application capable of being utilized on multiple 5G slices, it is possible to deploy multiple modular gateways on a single ros2 instance, and each modular gateway can transmit different topics. thus different topics can be transmitted at different QoS levels.
- DiscoveryService NAT traversal using STUN The current discovery service is designed for use in a lab environment where the entire 5G network shares a single LAN network. Thus it currently cannot easily handle different modular gateway being located behind NAT. This can be mitigated by implementing one of the recently popularised NAT traversal libraries such as ones utilising the STUN protocol. Due to the modular nature of all parts of this system adding the NAT traversal capabilities would only require changes to the networking parts and the discovery service connected parts, with the rest staying unmodified

- Further firmware and driver optimisation
 - Maximum movement execution time
 - Constant latency mode

3 Demo time

3.1 Hardware Demonstration

<https://youtu.be/R55aSlyh2z4>

3.2 P2P mode setup



Figure 2: P2P mode setup sequence diagram

3.3 P2P DS mode setup

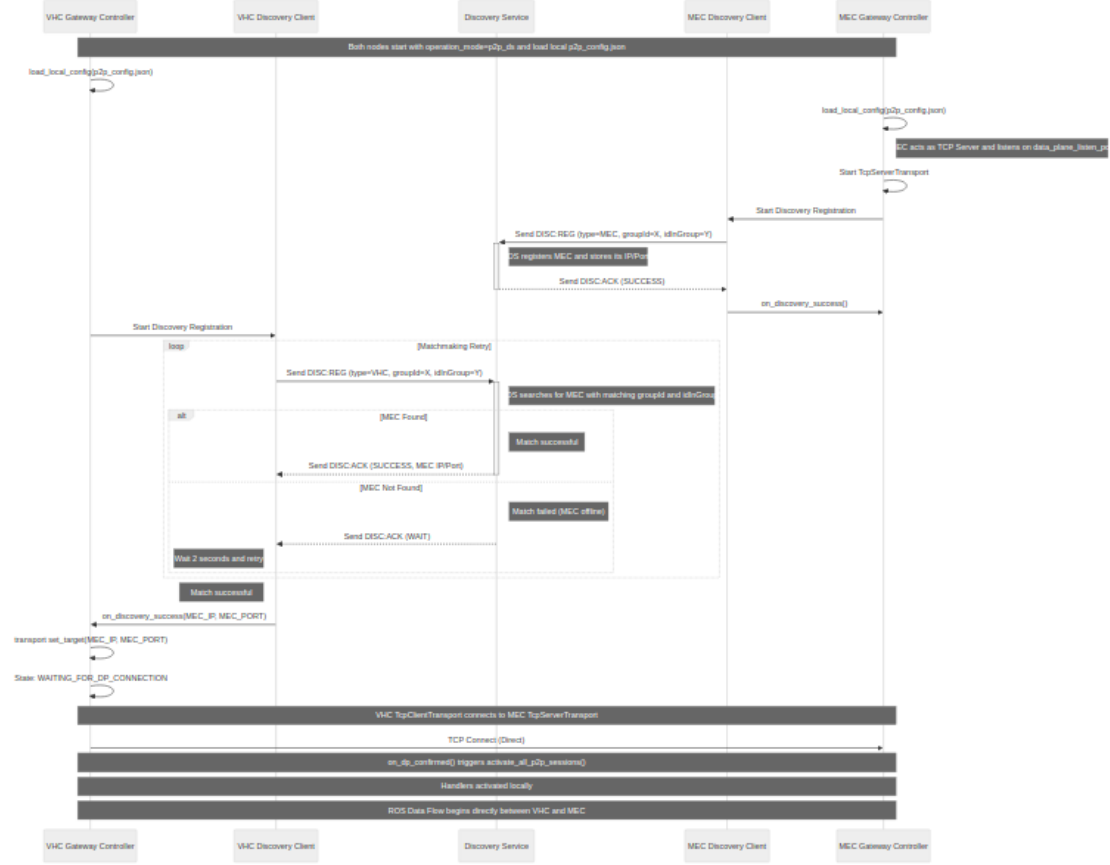


Figure 3: P2P DS mode setup sequence diagram